



Informe Técnico Breve ACTIVIDADES INDIVIDUALES



Estudiante: Paco Alanoca, Joaquin Nicolas
C.I.: 8354241

a) Clasificador de Texturas

Descripción

Aplicación de clasificación automática de regiones en imágenes mediante análisis estadístico por bloques utilizando NumPy. Cada bloque de 10×10 píxeles es evaluado con su promedio de color (RGB) y su varianza, asignando una etiqueta de categoría según umbrales definidos.

Pasos Realizados

1. Carga de imagen: El usuario sube una imagen desde el navegador. Se lee con FileReader y se decodifica en base64 para convertirla a un objeto PIL.Image en memoria.

2. Renderizado en canvas: La imagen original se dibuja en el primer <canvas> del DOM via PyScript.

3. Análisis por bloques: Se recorre la imagen en bloques (ventana) de 10×10 píxeles. Para cada bloque se calcula:

- Promedio RGB (`bloque.mean(axis=(0,1))`)
- Varianza del canal Rojo como indicador de rugosidad (`bloque.var(axis=(0,1))[0]`)

4. Clasificación por umbrales:

Clase	Criterio	Color asignado
Agua	Canal Azul dominante, varianza < 15 , verde < 100	Azul [0, 0, 255]
Vegetación	Canal Verde dominante, varianza > 20	Verde [0, 255, 0]
Zona Urbana	Varianza > 150 (alta dispersión)	Rojo [255, 0, 0]
Suelo desnudo	Caso base residual	Amarillo [255, 255, 0]

5. Renderizado del resultado: La imagen procesada (mapa de colores) se dibuja en el segundo <canvas>.

Herramientas y Tecnologías

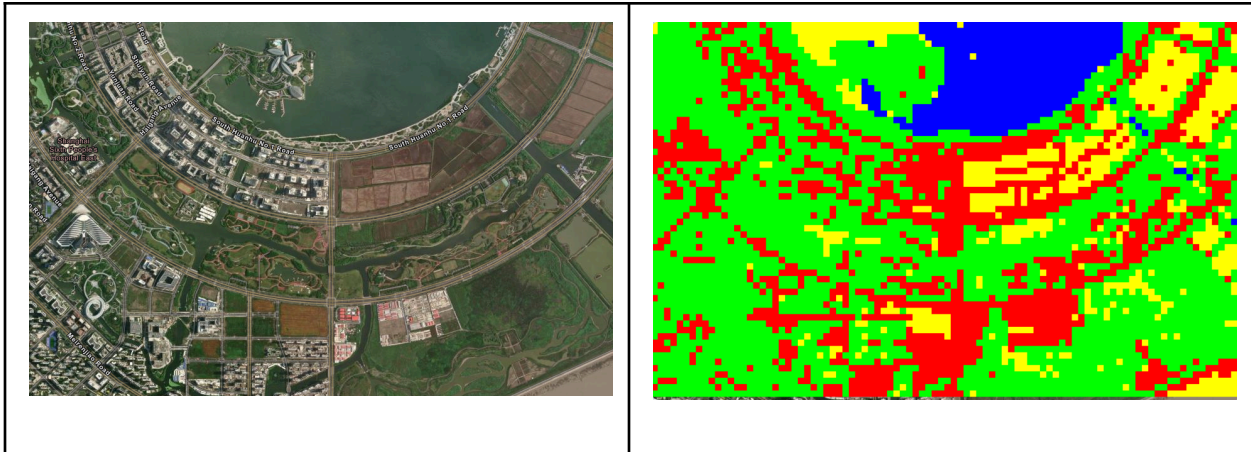
Herramienta	Uso
Python	Lenguaje de programación principal
PyScript 2024.1.1	Ejecución de Python directamente en el navegador
Pillow (PIL)	Carga y conversión de imágenes
NumPy	Operaciones vectorizadas sobre arrays de píxeles
HTML5 Canvas	Renderizado de imagen original y procesada

Capturas de Pantalla Ejecucion



Comparación Visual — Antes y Después

Antes: Imagen Original	Después: Imagen Clasificada por textura
------------------------	---



Interpretación del resultado: Las zonas azules representan agua, las verdes corresponden a vegetación o bosques, las rojas indican zonas urbanas de alta dispersión de píxeles, y las amarillas corresponden a suelo desnudo o zonas de transición.

Ejercicio b) Filtro de Suavizado Lineal 3×3

Descripción

Implementación de un filtro de media 3×3 (suavizado lineal) mediante convolución vectorizada con NumPy Slicing, sin usar bucles anidados. El filtro reduce el ruido en imágenes calculando el promedio aritmético de cada píxel con sus 8 vecinos.

Pasos Realizados

- 1. Carga de imagen:** Mismo mecanismo que el ejercicio anterior — FileReader → base64 → PIL.Image → numpy.array.
- 2. Renderizado de imagen original:** Se muestra en el canvas izquierdo antes del procesamiento.
- 3. Aplicación del filtro de media 3×3 mediante NumPy Slicing vectorizado:**
 - Se crean 9 rodajas (slices) de la matriz de la imagen, cada una desplazada un píxel en horizontal y vertical.
 - Se suman las 9 rodajas y se dividen entre 9.0 para obtener el promedio.
 - El resultado se escribe sobre los píxeles interiores [1:-1, 1:-1] del array, preservando los bordes originales.
 - La operación se aplica simultáneamente a los 3 canales (R, G, B).
- 4. Renderizado del resultado:** La imagen suavizada se dibuja en el canvas derecho.

Fórmula aplicada (por cada píxel interior [i,j])

$$\text{resultado}[i,j] = (I[i-1,j-1] + I[i-1,j] + I[i-1,j+1] + I[i,j-1] + I[i,j] + I[i,j+1] + I[i+1,j-1] + I[i+1,j] + I[i+1,j+1]) / 9$$

Herramientas y Tecnologías

Herramienta	Uso
Python	Lenguaje de programación principal
PyScript 2024.1.1	Ejecución de Python en el navegador
Pillow (PIL)	Carga y exportación de imágenes
NumPy	Convolución vectorizada via array slicing
HTML5 Canvas	Visualización del antes y después

Capturas de Pantalla Ejecucion



Comparación Visual — Antes y Después

Antes : Imagen Original	Después : Imagen con filtro
	

Interpretación del resultado: Los bordes de objetos y las transiciones bruscas se suavizan, reduciendo el ruido de alta frecuencia. La imagen resultante presenta tonos más uniformes. Esto es visible especialmente en zonas de textura fina.

Ejercicio c) Cover "La Vaca Lola"

Descripción

Script Python que procesa un video musical frame a frame, incrustando subtítulos sincronizados manualmente sobre cada fotograma usando OpenCV. El video resultante se exporta con los subtítulos quemados. Paralelamente, se desarrolló una animación visual complementaria con Godot Engine para las animaciones de los sprites.

Pasos Realizados

1. Carga del video fuente: Se abre assets/video/vacalola.mp4 con cv2.VideoCapture. Se obtienen los metadatos: FPS, ancho y alto de fotograma.

2. Configuración del escritor de salida: Se inicializa `cv2.VideoWriter` con el codec XVID para exportar el video procesado a `output/video-letra.mp4`.

3. Definición de subtítulos: Se crea una lista de tuplas (`tiempo_inicio`, `tiempo_fin`, `texto`) que cubre los versos y el coro de la canción, sincronizados manualmente con los tiempos del video.

4. Procesamiento frame a frame:

- Se lee cada fotograma con `cap.read()`.
- Se calcula el tiempo actual: `time = frame_index / fps`.
- Se busca en la lista de subtítulos si hay texto activo para ese instante.
- Si hay texto, se dibuja con `cv2.putText()` en la zona inferior del fotograma (posición `y = height - 80`), con fuente `HERSHEY_SIMPLEX`, tamaño 1.2 y color blanco con grosor 3 y suavizado anti-aliasing (`LINE_AA`).
- El fotograma (con o sin texto) se escribe al archivo de salida.

6. Unificación de Audio y Video: Se empleo de `ffmpeg` para la unificación del video y audio generado con IA

Notas:

Animación Godot: Se desarrolló una animación 2D con sprites animados de la vaca, fondo.

Audio Generado con IA: Se hizo uso de Suno para la generación de la pista de audio estilo musica Ranchera, con una resescritura de la cancion de la vaca lola.

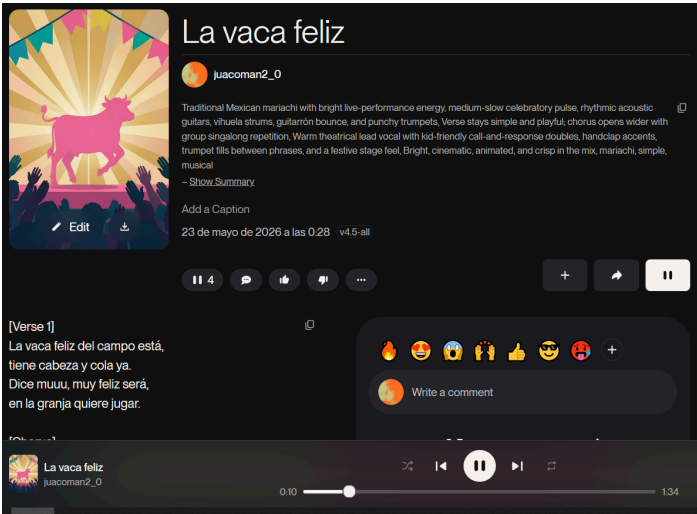
Herramientas y Tecnologías

Herramienta	Uso
Python	Lenguaje de programación principal
OpenCV (cv2)	Lectura, procesamiento y escritura de video frame a frame
Godot Engine 4	Motor de videojuegos para animación complementaria
GDScript	Lógica de animación y sincronización en Godot
Suno AI	Herramienta de creación musical por inteligencia artificial

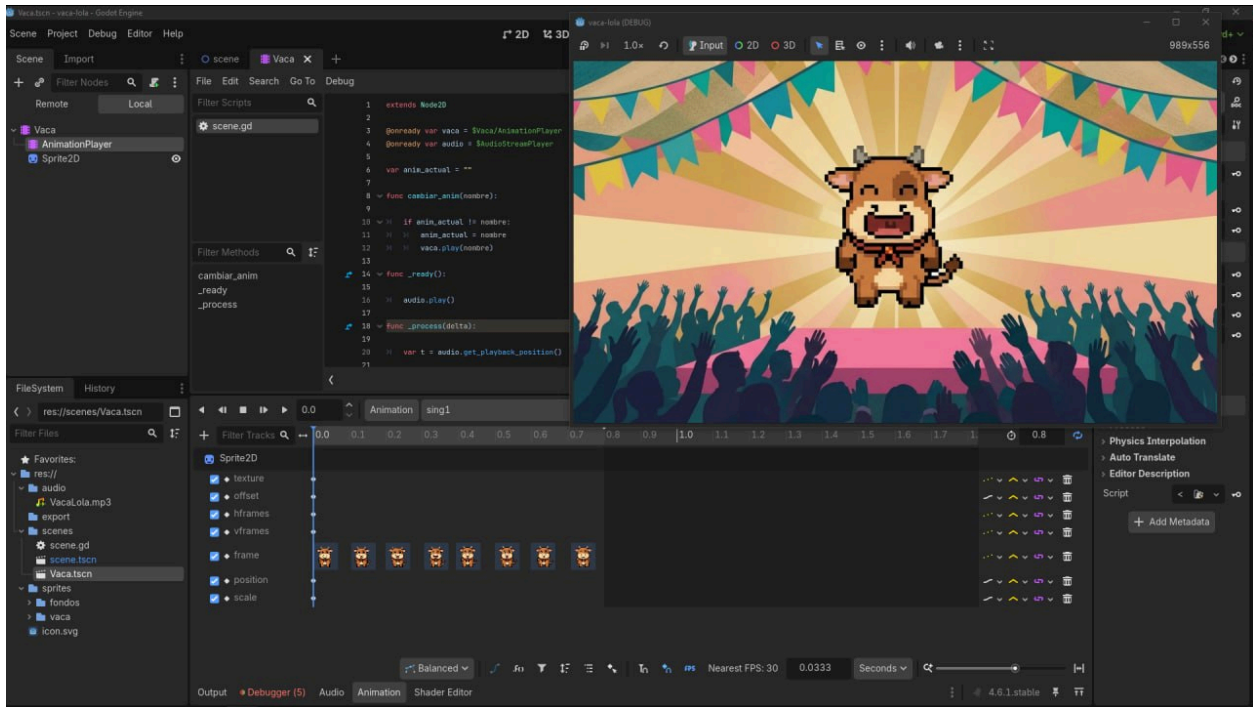
ffmpeg	Colección de software libre para unificar audio y vídeo.
--------	--

Capturas de Pantalla

SunoAI



Godot Engine



Resultado Final (Subtítulos + Animacion + Audio)

Frames del resultado	Frames del resultado
 Dice muuu, muy feliz sera	 Muu muu muu, puede bailar

Interpretación del resultado: El script genera un video nuevo donde cada línea del coro y los versos aparece sincronizada en el momento correcto, permitiendo visualizar la letra de la canción como karaoke.